

Guardrails Under Retrieval-Mediated Attacks: Evaluating Placement, Architecture, and Robustness in RAG Systems

Taixing Bi
Independent Researcher
New York, NY, USA
taixingbijob@gmail.com

Abstract—Retrieval-augmented generation (RAG) improves large language models by grounding generation in external knowledge [1–4], but retrieval also creates an attack surface through which adversarial instructions and poisoned evidence can enter the model context [5–8]. Prior work has demonstrated direct and indirect prompt injection [9–14], retrieval poisoning [5,6], and adversarial manipulation of LLM-integrated applications [10–13]. Meanwhile, guardrail systems increasingly combine dedicated safety models, programmable controls, structured instruction handling, and input/output filtering [15–22]. However, it remains unclear where guardrails should be placed in a RAG pipeline, how they should be composed, and whether apparent robustness persists under previously unseen or adaptive attacks.

We present a systematic empirical study of guardrails under four retrieval-mediated adversarial conditions across multiple target LLMs. We study three dimensions of guardrail design: placement (query, retrieved context, output, and context-plus-output), composition (deterministic rules, LLM-based moderation, and hybrid defenses), and guard-model capacity. We further evaluate frozen defenses under unseen attack formulations, adaptive attackers, cross-model transfer, and cross-dataset transfer.

Guardrail placement strongly determines the safety–utility tradeoff. Query-only protection leaves a 70.3% safety attack success rate (ASR), whereas guarding retrieved context before generation reduces observed in-distribution ASR to 0% while maintaining 86.3% attacked-task accuracy. Guard-model scale does not monotonically predict robustness. More importantly, apparent robustness against known attacks does not fully generalize: a single-stage context defense increases from 0% observed ASR on known attacks to 9.3% on unseen formulations and 32.7% against adaptive attacks. Adding output defense restores 0% observed ASR in these evaluations, but incurs additional utility and latency costs.

Our results suggest that robust RAG guardrails should be treated as a systems-security problem involving trust boundaries, defense placement, architecture, and adversarial distribution shift rather than primarily as a guard-model scaling problem.

Keywords—retrieval-augmented generation, guardrails, prompt injection, adversarial robustness, LLM security, RAG safety

I. INTRODUCTION

Retrieval-augmented generation has become a common architecture for grounding large language models in external knowledge [1]. It combines parametric generation with retrieved evidence, building on advances in dense retrieval [2], knowledge-intensive language processing [3], and modern retrieval-augmented language modeling [1–4]. RAG is particularly useful for applications that require private, domain-specific, or recently updated information.

Retrieval, however, introduces an additional security boundary. Retrieved documents originate outside the generator and should not automatically inherit the authority of system or developer instructions. Prior work has demonstrated that adversarial instructions embedded in external content can compromise LLM-integrated applications even when the user's own query is benign [9–13]. RAG systems further expose their knowledge corpus to poisoning: an attacker capable of introducing malicious documents may manipulate retrieval and steer generation toward attacker-selected outputs [5,6].

These threats arise partly because instructions and untrusted data are represented through the same natural-language interface. Existing defenses therefore explore structured separation between instructions and data, explicit instruction hierarchies, transformations that highlight untrusted content, and preference-based alignment against prompt injection [19–22]. Practical security guidance similarly identifies indirect prompt injection and external-content manipulation as major risks for LLM applications [23,24].

Yet a RAG application provides several possible locations for defensive intervention. A guardrail can inspect the user query, retrieved documents, generated output, or multiple stages. Dedicated safety models and programmable guardrail systems support such runtime controls [15–18,25], but their effectiveness depends not only on detection capability but also on where the defense is positioned relative to the attack surface.

This motivates our central research question:

Where should guardrails be placed in a RAG pipeline, how should they be composed, and how robust are they when

attackers move beyond patterns observed during defense development?

We address this question through a controlled evaluation of four retrieval-mediated adversarial conditions: indirect prompt injection, policy-violating retrieved content, context poisoning, and authority-conflicting instructions. The first, second, and fourth conditions test instruction-mediated security failures, while context poisoning evaluates integrity failures caused by misleading retrieved evidence [5,7,8].

We first study guardrail placement, comparing defenses applied to the user query, retrieved context before generation, generated output, and both retrieved context and generated output. We then examine guardrail composition, comparing deterministic rules, LLM-based moderation, and hybrid defenses. Finally, we vary guard-model capacity and evaluate whether model scaling provides consistent improvements in security.

Crucially, we do not restrict evaluation to attack structures represented during defense development. Static adversarial evaluation can substantially overestimate robustness when defenses are exposed to unfamiliar or adaptive attacks [26,27]. Modern LLM security benchmarks similarly emphasize diverse and automated adversarial evaluation [28–31]. We therefore freeze the defense and evaluate it against unseen formulations generated independently from the original attack templates and against adaptive attacks designed to preserve the malicious objective while avoiding recognizable defensive triggers.

Our experiments reveal a substantial gap between in-distribution security and adversarial robustness. Context-level protection provides the strongest safety–utility tradeoff on known attacks, reducing observed Safety ASR to 0% while retaining 86.3% attacked-task accuracy. Query-only protection leaves 70.3% ASR. However, the same single-stage context defense degrades from 0% on known attacks to 9.3% on unseen formulations and 32.7% against adaptive attacks. Adding output protection restores 0% observed ASR in these evaluations, although at additional utility and latency cost.

Our main contributions are:

- 1. Retrieval-mediated attack evaluation.** We systematically evaluate four adversarial conditions delivered through retrieved context rather than restricting security evaluation to malicious user prompts.
- 2. Guardrail placement analysis.** We compare query, context, output, and context-plus-output defenses and show that context-level protection provides the strongest in-distribution safety–utility tradeoff in our experiments.
- 3. Composition and capacity analysis.** We compare deterministic, LLM-based, and hybrid defenses across guard-model capacities and find that composition drives the largest change in Safety ASR, while guard-model scale primarily affects attacked-task accuracy rather than robustness.
- 4. Adversarial generalization evaluation.** We evaluate frozen defenses under unseen formulations, adaptive attacks, multiple target models, and cross-dataset transfer, showing that strong in-distribution results can substantially overestimate robustness.

II. BACKGROUND AND THREAT MODEL

A. Retrieval-Augmented Generation

RAG conditions generation on evidence obtained from an external retrieval system [1]. Dense Passage Retrieval established a strong neural retrieval approach for open-domain QA [2], while KILT standardized evaluation across knowledge-intensive language tasks [3]. RAG evaluation has subsequently expanded beyond answer correctness to dimensions such as relevance, faithfulness, and retrieval quality [32,33].

Our primary experiments use HotpotQA [34], with SQuAD [35] used for frozen-defense cross-dataset evaluation.

B. Retrieval-Mediated Attacks

Prompt injection attempts to alter the behavior of an instruction-following model using attacker-controlled text [9,14]. Indirect prompt injection extends this threat to content obtained from external resources [10–13]. Consequently, a user can issue an entirely benign query while the malicious payload enters through a retrieved document.

RAG is particularly exposed because retrieved passages are directly incorporated into the generator context. PoisonedRAG demonstrates that malicious documents inserted into a retrieval corpus can steer generated answers [5], while related work has explored poisoning of external memories and knowledge bases used by LLM systems [6].

We evaluate four conditions:

- **Indirect prompt injection:** retrieved text attempts to redirect generation away from the legitimate task.
- **Policy-violating retrieval:** retrieved content attempts to induce behavior inconsistent with the intended system policy.
- **Context poisoning:** retrieved evidence is modified to promote an incorrect answer.
- **Authority conflict:** retrieved content impersonates privileged instructions such as system or administrator messages.

The attacker can influence content entering the retrieval corpus but cannot modify the trusted system prompt, defense implementation, or target model parameters.

C. Known, Unseen, and Adaptive Attacks

We distinguish three adversarial settings.

Known attacks follow structures represented during defense development.

Unseen attacks preserve the malicious objective while using held-out formulations independently generated from the development templates.

Adaptive attacks assume knowledge that a guardrail exists and explicitly attempt to preserve the malicious objective while avoiding recognizable defensive triggers.

This distinction follows a fundamental principle from adversarial-security evaluation: defenses should be tested against attacks adapted to or independent from the structures used during defense construction [26,27].

III. GUARDRAIL DESIGNS

Guardrail systems can intervene at several layers of an LLM application. Programmable systems such as NeMo Guardrails expose multiple runtime controls [15], while dedicated safety models such as Llama Guard [16], ShieldGemma [17], WildGuard [18], and Granite Guardian [25] classify potentially unsafe prompts or responses.

We study these mechanisms as components of a RAG architecture rather than evaluating a new standalone safety classifier.

A. Placement

We evaluate four placements:

- **Query-only (Q):** the guardrail evaluates the user request before retrieval.
- **Context (C):** the guardrail evaluates retrieved evidence after retrieval and before generation.
- **Output-only (O):** the generator processes the retrieved context normally, after which the guardrail evaluates the generated response.
- **Context + Output (CO):** the system evaluates both retrieved context and generated output.

This comparison tests whether aligning the defense with the location of untrusted content improves security and utility.

B. Composition

We compare three defense compositions.

The rule-based defense performs deterministic, pattern-based detection and sanitization: a fixed set of regular-expression signatures is matched against each line of retrieved context, matching lines are dropped, and the remaining lines are kept as sanitized evidence; if too little text remains after dropping, the document is blocked outright rather than passed through empty. This mechanism has no semantic understanding — it recognizes recurring surface patterns rather than the meaning of the instruction, so it is precise on known phrasings but blind to unfamiliar ones (Section V-C).

The LLM-based defense uses a guard model to classify retrieved content and determine whether it should be allowed, sanitized, or blocked, based on a semantic judgment of intent rather than fixed surface patterns.

The hybrid defense combines the two: deterministic pattern rules handle high-confidence attack structures at low cost, while the guard model evaluates residual cases requiring semantic interpretation.

This design is related to programmable guardrail architectures [15], but our goal is to evaluate the interaction between pattern-based and semantic defenses under retrieval-mediated attacks.

C. Guard-Model Capacity

Dedicated safety models span a broad range of parameter counts [16–18,25]. We therefore evaluate multiple guard-model capacities while keeping the surrounding architecture and evaluation conditions fixed.

This allows us to test whether larger guard models reliably provide stronger security.

IV. EXPERIMENTAL SETUP

A. Dataset and Attack Conditions

Our primary evaluation uses 100 HotpotQA [34] seeds. For each seed we construct four retrieval-mediated attack conditions — indirect prompt injection (A1), context poisoning (A2), policy-violating retrieval (A3), and authority conflict (A4) — yielding 400 validated attack cases, where a case is accepted only if it passes both a semantic-validity check and a payload-delivery check. Table I summarizes the dataset.

Table I. Adversarial evaluation dataset.

Attack condition	Validated cases
A1 — Indirect prompt injection	100
A2 — Context poisoning	100
A3 — Policy-violating retrieval	100
A4 — Authority conflict	100

B. Guardrail Configurations and Target Models

We evaluate three guardrail configurations that compose placement and architecture: G0 (no guardrail), G1 (hybrid input/context defense only), and G2 (hybrid input/context defense plus an output defense). For placement experiments (Section V-A) we use four placements — Q (query), C (context), O (output), and CO (context+output); in the main robustness experiments (Sections V-C–V-E), G1 corresponds to the context-only hybrid configuration (C), while G2 corresponds to the context-plus-output hybrid configuration (CO). Unless otherwise noted, the input defense uses our frozen hybrid composition — deterministic rules evaluated first, with an LLM guard (gpt-oss) resolving residual cases — under a fixed prompt version (v3). Target generation models are nova-pro and llama; the guard and judge model is gpt-oss unless we are explicitly varying guard-model capacity (Section V-B).

C. Metrics

We report Safety Attack Success Rate (Safety ASR), the mean attack-success rate over the three instruction-mediated conditions (A1, A3, A4). Context poisoning (A2) is scored separately as Poisoning Success Rate (PSR), since it is an integrity failure rather than an instruction-following failure. Macro success rate averages all four conditions. We additionally report attacked-task accuracy (Acc), benign accuracy, over-refusal rate, and, where tracked, mean end-to-end latency and guard-model call counts. All intervals are 95% percentile bootstrap confidence intervals (n = 2000 resamples).

D. Known, Unseen, and Adaptive Protocol

For known-attack evaluation, guardrails are evaluated in-distribution on attack templates matched to defense development. For unseen-attack evaluation (Experiment 4), the defense is frozen and evaluated on 50 held-out seeds using

A1/A3/A4 templates generated independently by a different generator model (deepseek); context poisoning is excluded from this transfer check since it is an integrity concern rather than a template-matching concern. For adaptive-attack evaluation, the same frozen defense is evaluated against attacks explicitly constructed to avoid known rule triggers while preserving the underlying malicious objective. Cross-model transfer repeats the main protocol on an additional target model (deepseek); cross-dataset transfer repeats it on SQuAD [35] seeds without retuning the frozen defense.

V. RESULTS

A. Placement Matters

Placement produces substantial differences in both security and utility. Table II reports Safety ASR, PSR, and attacked-task accuracy for each of the four placements, evaluated on 50 seeds $\times$ 4 attacks $\times$ 2 target models (1,600 attack conditions).

Table II. Guardrail placement results (50 seeds, 2 target models).

Placement	Safety ASR ↓	PSR ↓	Acc ↑
Query-only (Q)	0.703 [0.653, 0.757]	0.550 [0.450, 0.650]	0.258 [0.215, 0.305]
Output-only (O)	0.013 [0.003, 0.027]	0.560 [0.470, 0.650]	0.278 [0.235, 0.323]
Context (C)	0.000 [0.000, 0.000]	0.260 [0.180, 0.350]	0.863 [0.830, 0.895]
Context+Output (CO)	0.000 [0.000, 0.000]	0.390 [0.300, 0.490]	0.840 [0.805, 0.873]

Query-only protection leaves a Safety ASR of 70.3% and attacked-task accuracy of 25.8%. Because the adversarial payload enters after query inspection, the guardrail often receives no malicious signal.

Output-only protection reduces Safety ASR to 1.3%, but attacked-task accuracy remains only 27.8%. Output filtering can suppress unsafe final responses, but it acts after adversarial context has already influenced generation.

Context-level protection reduces observed in-distribution Safety ASR to 0% while achieving 86.3% attacked-task accuracy. Context-plus-output protection also achieves 0% observed ASR, with 84.0% attacked-task accuracy and additional inference overhead.

These results indicate that post-retrieval, pre-generation protection provides the strongest in-distribution safety–utility tradeoff among the evaluated placements.

B. Composition Dominates Safety Robustness, While Scale Primarily Affects Utility

Composition experiments reveal distinct safety–utility characteristics. Table III compares three input-defense compositions — hybrid, rules-only, and LLM-only — under the context placement (G1), on the same 100-seed main evaluation.

Table III. Guardrail composition (context placement, G1).

Composition	Safety ASR ↓	PSR ↓	Acc ↑
Hybrid	0.000 [0.000, 0.000]	0.300 [0.235, 0.360]	0.820 [0.794, 0.846]
Rules-only	0.000 [0.000, 0.000]	0.415 [0.345, 0.480]	0.786 [0.757, 0.814]
LLM-only	0.457 [0.417, 0.497]	0.335 [0.270, 0.405]	0.393 [0.359, 0.426]

Deterministic rules alone already drive Safety ASR to 0% on known instruction-mediated attacks, but LLM-only composition leaves Safety ASR at 45.7% — the guard model alone misses a large share of recognizable attack structures. Hybrid composition matches rules-only on Safety ASR while improving PSR (30.0% vs. 41.5%) and attacked-task accuracy (82.0% vs. 78.6%). These results suggest that semantic evaluation of residual cases can complement deterministic pattern-based filtering.

Table IV then fixes the hybrid composition and varies guard-model capacity from 3B to 120B parameters, under both known (in-distribution) and unseen attacks, to isolate the effect of scale from the effect of architecture.

Table IV. Guard-model capacity under hybrid context defense (G1).

Guard size	Known ASR	Known Acc	Unseen ASR
3B (minstral)	0.000 [0.000, 0.000]	0.895 [0.850, 0.935]	0.047 [0.023, 0.070]
14B (minstral)	0.000 [0.000, 0.000]	0.835 [0.780, 0.885]	0.100 [0.067, 0.137]
70B (llama)	0.000 [0.000, 0.000]	0.890 [0.845, 0.930]	0.103 [0.073, 0.140]
120B (gpt-oss, main)	0.000 [0.000, 0.000]	0.820 [0.794, 0.846]	0.093 [0.063, 0.127]

Under hybrid composition, Safety ASR does not decrease monotonically with model scale: all four guard sizes reach 0% in-distribution ASR, while unseen-attack ASR is lowest for the smallest guard (3B, 4.7%) rather than the largest (120B, 9.3%). Scale instead moves attacked-task accuracy on unseen attacks, which rises from 54.0% (3B) to 83.0% (120B, gpt-oss) — see Section V-C.

Taken together, Tables III and IV isolate the two design axes along a single, consistent metric. Composition (hybrid vs. rules-only vs. LLM-only) drives the largest change in Safety ASR (0% to 45.7%). Guard-model capacity under a fixed hybrid composition, by contrast, drives essentially no change in known-attack Safety ASR — all four sizes reach 0% — and only a modest, non-monotonic change under unseen attacks; its main effect instead falls on attacked-task accuracy, which rises from 54.0% (3B) to 83.0% (120B). We therefore do not treat these as a controlled factorial comparison of "architecture vs. scale" in general, but within our evaluated settings, composition is the factor that determines whether Safety ASR is near zero or substantial, while scale is the factor that determines how much task utility the defense preserves. Our results do not support the

assumption that a larger guard model automatically creates a stronger RAG security boundary on its own.

C. Known Attacks Overestimate Robustness

The most important change occurs under attack distribution shift. Table V and Figure 1 track Safety ASR for a single-stage context defense (G1) versus a defense-in-depth context-plus-output configuration (G2/CO) as the evaluation moves from known to unseen to adaptive attacks.

Table V. Safety ASR under distribution shift.

Condition	Context only (G1)	Context+Output (G2)
Known	0.000 [0.000, 0.000]	0.000 [0.000, 0.000]
Unseen	0.093 [0.063, 0.127]	0.000 [0.000, 0.000]
Adaptive	0.327 [0.277, 0.380]	0.000 [0.000, 0.000]

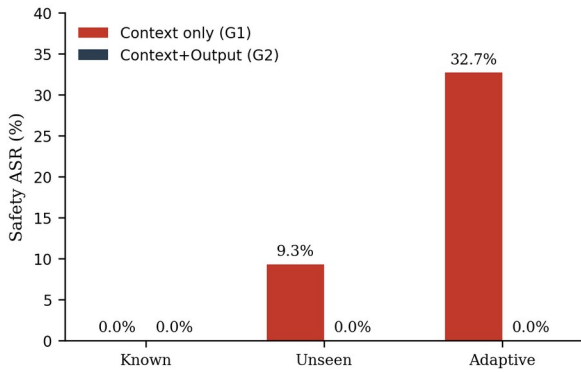


Figure 1. Safety ASR for a single-stage context defense (G1) vs. a defense-in-depth context+output defense (G2) across known, unseen, and adaptive attacks.

For known attacks, the context/input defense achieves 0% observed Safety ASR. After freezing the defense and evaluating unseen formulations generated from held-out attack structures and a different generator, Safety ASR rises to 9.3%. Under adaptive attacks explicitly designed to avoid recognizable defensive triggers, Safety ASR reaches 32.7%. Context-plus-output protection (G2), in contrast, remains at 0% observed Safety ASR across all three regimes.

This progression demonstrates that strong performance against familiar attacks can substantially overestimate adversarial robustness. The finding is consistent with the broader security principle that defenses should be evaluated against adaptive adversaries rather than static attack sets alone [26,27].

D. Defense-in-Depth Under Distribution Shift

Against known attacks, context-only and context-plus-output defenses both achieve 0% observed Safety ASR. Under these conditions, the output layer appears largely redundant.

This conclusion changes under distribution shift.

For unseen attacks, context/input protection leaves 9.3% ASR, whereas context-plus-output protection reduces observed ASR to 0%.

Under adaptive attacks, context/input protection reaches 32.7% ASR, while context-plus-output protection again achieves 0% observed ASR in our evaluation.

Output protection therefore functions as a defense-in-depth layer. Its marginal value is limited against familiar attacks but increases substantially when the earlier context defense is bypassed.

This additional protection comes with increased inference latency and computational cost. The result exposes a security–utility–latency tradeoff rather than implying that additional guardrail layers are universally preferable.

E. Cross-Model and Cross-Dataset Transfer

Target models exhibit substantially different unprotected susceptibility to retrieval-mediated attacks, indicating that intrinsic model robustness varies across model families.

Nevertheless, context-level protection consistently reduces observed attack success across the evaluated targets.

We further freeze the defense developed on HotpotQA and evaluate it on SQuAD without retuning. On SQuAD, unprotected Safety ASR is 46.0%, while context and context-plus-output protection both achieve 0% observed Safety ASR. Attacked-task accuracy remains approximately 79–82%.

These results provide evidence that the main placement and architecture observations are not restricted to a single target model or QA benchmark.

VI. DISCUSSION

A. Retrieved Context Is a Trust Boundary

Retrieved documents should be treated as untrusted external input rather than automatically trusted knowledge.

Indirect prompt-injection research demonstrates that external resources can become instruction channels despite lacking legitimate authority [10–13]. Security guidance similarly identifies remote and indirect prompt injection as important risks in LLM applications [23,24].

Our placement results provide empirical evidence of the architectural consequence: protecting the user query is insufficient when the attack enters downstream through retrieval.

B. Zero Observed ASR Is Not a Security Guarantee

Several configurations achieve 0% observed attack success under specific experimental conditions. These measurements should not be interpreted as proofs of universal attack prevention.

The unseen and adaptive experiments demonstrate this directly. A defense that appears perfect against known attack structures can degrade substantially when the adversary changes its formulation.

This mirrors a long-standing lesson from adversarial machine learning: robustness claims should be evaluated against attacks appropriate to the actual defense [26,27].

We therefore explicitly report observed ASR under the evaluated threat model.

C. Guard Models Are Components, Not Complete Security Boundaries

Increasing guard-model size is not a reliable substitute for security architecture.

A guard model operates inside a larger application. Its effectiveness depends on what information it receives, when it receives that information, what authority its decision has, and how the system responds to the decision.

Structured queries [19], instruction hierarchy [20], spotlighting [21], and preference-based prompt-injection defenses [22] similarly motivate stronger separation between trusted instructions and untrusted content.

Our results suggest that placement and composition are the dominant factors for safety robustness, while raw guard-model capacity primarily shifts task utility rather than security.

D. Security, Utility, and Latency Must Be Evaluated Together

A system could trivially reduce attack success by refusing every request. Such a defense would be secure under a narrow attack metric but unusable.

Safety evaluation should therefore account for legitimate task performance and over-refusal [18,30,36].

Our results extend this principle to RAG guardrails by jointly measuring attack success, attacked-task accuracy, benign behavior, and runtime overhead.

The appropriate architecture consequently depends on the deployment threat model. Context-only protection provides a strong tradeoff under the evaluated in-distribution workload, whereas higher-risk environments may justify context-plus-output defense despite additional latency and utility costs.

VII. RELATED WORK

A. RAG, Retrieval Poisoning, and Prompt Injection

RAG combines retrieval with generation to condition model responses on external evidence [1], building on DPR [2] and KILT [3]. RAG evaluation frameworks such as RAGAS [32] and ARES [33] subsequently expanded evaluation beyond answer accuracy to faithfulness and context relevance, and related work has studied robustness to noisy or misleading retrieved evidence [8]. Prompt injection exploits the inability of instruction-following models to separate attacker-controlled text from legitimate instructions [9,14]; indirect prompt injection extends this to externally sourced content [10], with BIPIA [11], InjecAgent [12], and AgentDojo [13] extending the threat to tool-integrated and agentic settings. PoisonedRAG [5] and AgentPoison [6] show that a retrieval corpus or external memory can itself be poisoned to steer generation. Our work is complementary to all of these: rather than proposing a new retriever, evaluation metric, or poisoning attack, we study how runtime guardrails should be positioned and composed once the retrieval channel is treated as an adversarial surface.

B. Prompt-Injection and Guardrail Defenses

Several approaches strengthen separation between trusted instructions and untrusted data: StruQ [19] uses structured queries, Instruction Hierarchy [20] trains models to prioritize privileged instructions, Spotlighting [21] transforms untrusted content to mark its lower trust status, and SecAlign [22] uses preference optimization. Separately, programmable and model-based guardrail systems — NeMo Guardrails [15], Llama Guard [16], ShieldGemma [17], WildGuard [18], and Granite Guardian [25] — provide runtime detection and moderation. Our work is complementary to both lines: rather than proposing another model-training defense or a new guard classifier, we study how existing runtime defenses should be placed and composed within a RAG pipeline, and how guard-model capacity trades off against architecture.

C. Adaptive Security Evaluation

Static attacks can produce misleading robustness estimates: Athalye et al. showed several adversarial defenses failed under appropriately adapted attacks [26], and Tramèr et al. emphasized explicitly adaptive evaluation [27]. Recent LLM security benchmarks — HarmBench [28], JailbreakBench [29], StrongREJECT [30], and related red-teaming frameworks [31] — apply similar principles to LLM safety. Our known–unknown–adaptive protocol applies this evaluation discipline specifically to RAG guardrails.

VIII. LIMITATIONS

Our evaluation covers a finite set of attack classes, target models, guard models, and QA datasets. Consequently, 0% observed ASR does not establish universal security.

The evaluated attacks do not represent all threats to production RAG systems. Real deployments may contain multi-turn interaction, agents, tool execution, memory, multimodal documents, dynamic indexes, and heterogeneous trust levels. Agent-oriented security benchmarks demonstrate that these additional components create further indirect-injection surfaces [12,13].

Our deterministic components are necessarily sensitive to attack structure. Although unseen and adaptive evaluations reduce the risk of reporting purely template-specific performance, stronger attackers may discover additional bypasses.

Cross-dataset evaluation currently covers QA benchmarks rather than heterogeneous enterprise domains. Future work should evaluate frozen defenses across broader document distributions and production retrieval architectures.

Finally, latency and cost depend on deployment infrastructure and model provider. We therefore interpret overhead primarily as a relative tradeoff among defense configurations rather than as a universal deployment estimate.

IX. CONCLUSION

We systematically evaluate guardrails against attacks delivered through retrieved context in RAG systems.

Our results show that guardrail effectiveness depends strongly on where defenses are placed and how they are composed. Guarding retrieved context before generation provides the strongest in-distribution safety–utility tradeoff among the evaluated placements, while increasing guard-model capacity does not consistently improve robustness.

Most importantly, apparent robustness against familiar attacks does not imply robustness to changing adversaries. A single-stage context defense increases from 0% observed ASR on known attacks to 9.3% on unseen formulations and 32.7% under adaptive attacks. Defense-in-depth restores 0% observed ASR in these evaluated settings but incurs additional utility and latency costs.

These findings reinforce the need to treat retrieved context as untrusted input [10–13,23,24] and to evaluate security using attacks independent of or adapted to the defense [26–30].

Robust RAG guardrails depend more on defense architecture, placement, and adversarial evaluation than on simply scaling the guard model.

REFERENCES

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.
- [2] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," *EMNLP*, 2020.
- [3] F. Petroni et al., "KILT: A Benchmark for Knowledge Intensive Language Tasks," *NAACL*, 2021.
- [4] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv*, 2023.
- [5] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models," *USENIX Security*, 2025.
- [6] X. Chen et al., "AgentPoison: Red-Teaming LLM Agents via Poisoning Memory or Knowledge Bases," *NeurIPS*, 2024.
- [7] J. Chen et al., "Benchmarking Large Language Models in Retrieval-Augmented Generation," *AAAI*, 2024.
- [8] X. Wang et al., "RECALL: A Benchmark for LLM Robustness against External Counterfactual Knowledge," 2024.
- [9] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques for Language Models," 2022.
- [10] K. Greshake et al., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," 2023.
- [11] J. Yi et al., "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," *KDD*, 2025.
- [12] Q. Zhan et al., "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," *ACL*, 2024.
- [13] E. Debenedetti et al., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," *NeurIPS*, 2024.
- [14] P. Toyer et al., "Tensor Trust: Interpretable Prompt Injection Attacks from an Online Game," *ICLR*, 2024.
- [15] T. Rebedea et al., "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," *EMNLP System Demonstrations*, 2023.
- [16] H. Inan et al., "Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations," 2023.
- [17] Google, "ShieldGemma: Generative AI Content Moderation Based on Gemma," 2024.
- [18] S. Han et al., "WildGuard: Open One-Stop Moderation Tools for Safety Risks, Jailbreaks, and Refusals of LLMs," *NeurIPS*, 2024.
- [19] S. Chen et al., "StruQ: Defending Against Prompt Injection with Structured Queries," *USENIX Security*, 2025.
- [20] E. Wallace et al., "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," 2024.
- [21] K. Hines et al., "Defending Against Indirect Prompt Injection Attacks with Spotlighting," 2024.
- [22] S. Chen et al., "SecAlign: Defending Against Prompt Injection with Preference Optimization," *CCS*, 2025.
- [23] OWASP, "LLM01: Prompt Injection," *OWASP Top 10 for LLM Applications*, 2025.
- [24] NIST, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, 2024.
- [25] IBM Research, "Granite Guardian," 2024.
- [26] A. Athalye, N. Carlini, and D. Wagner, "Obfuscated Gradients Give a False Sense of Security: Circumventing Defenses to Adversarial Examples," *ICML*, 2018.
- [27] F. Tramèr et al., "On Adaptive Attacks to Adversarial Example Defenses," *NeurIPS*, 2020.
- [28] M. Mazeika et al., "HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal," *ICML*, 2024.
- [29] P. Chao et al., "JailbreakBench: An Open Robustness Benchmark for Jailbreaking Large Language Models," *NeurIPS*, 2024.
- [30] A. Souly et al., "A StrongREJECT for Empty Jailbreaks," *NeurIPS*, 2024.
- [31] M. Samvelyan et al., "Rainbow Teaming: Open-Ended Generation of Diverse Adversarial Prompts," *NeurIPS*, 2024.
- [32] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," *EACL System Demonstrations*, 2024.
- [33] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," *NAACL*, 2024.
- [34] Z. Yang et al., "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," *EMNLP*, 2018.
- [35] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," *EMNLP*, 2016.
- [36] P. Röttger et al., "XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models," *NAACL*, 2024.
- [37] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, 1975.
- [38] NIST, "Zero Trust Architecture," NIST SP 800-207, 2020.